



DESARROLLO DE SOFTWARE MULTIPLATAFORMA

Desarrollo Web Profesional

CERVANTES ROMERO ABRAHAM

8º CUATRIMESTRE GRUPO "A"

Arquitectura de autenticación y control de roles

1. ¿Qué es autenticación y en qué se diferencia de autorización?

En los sistemas informáticos existen dos procesos fundamentales para controlar el acceso a los recursos: **autenticación** y **autorización**.

Autenticación

La **autenticación** es el proceso mediante el cual un sistema **verifica la identidad de un usuario**.

En otras palabras, responde a la pregunta:

¿Quién eres?

Se realiza normalmente mediante:

- Usuario y contraseña
- Token de acceso
- Autenticación biométrica
- Autenticación de dos factores (2FA)

Ejemplo:

Un usuario introduce:

correo: usuario@email.com
contraseña: *****

El servidor verifica esos datos en la base de datos.
Si coinciden, el usuario queda **autenticado**.

Autorización

La **autorización** ocurre después de la autenticación y define **qué acciones puede realizar el usuario dentro del sistema**.

Responde a la pregunta:

¿Qué puedes hacer dentro del sistema?

Ejemplo:

Un usuario autenticado puede:

- Ver información
- Crear registros
- Editar datos
- Eliminar contenido

Pero no todos los usuarios pueden hacer lo mismo.

Ejemplo:

Usuario	Permiso
Cliente	Ver productos
Empleado	Gestionar pedidos
Administrador	Gestionar usuarios

Diferencia clave

Concepto	Pregunta que responde
Autenticación	¿Quién eres?
Autorización	¿Qué puedes hacer?

2. ¿Qué son JWT, sesiones y OAuth?

JWT (JSON Web Token)

JWT es un **mecanismo de autenticación basado en tokens**.

Cuando el usuario inicia sesión correctamente, el servidor genera un **token firmado** que contiene información del usuario.

Ejemplo de contenido de un JWT:

```
{
  user_id: 15,
  role: "admin",
  exp: 1710000000
}
```

Ese token se envía al cliente y el cliente lo incluye en cada petición al servidor.

Ejemplo:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Ventajas

- Escalable
- No requiere guardar sesiones en el servidor
- Muy usado en APIs REST

Desventajas

- Si se roba el token puede usarse hasta que expire
- Difícil invalidarlo antes de tiempo

Sesiones

Las **sesiones** son un método tradicional de autenticación.

Proceso:

1. El usuario inicia sesión.
2. El servidor crea una sesión en la base de datos.
3. El servidor envía un **session ID** al navegador en una cookie.
4. Cada petición incluye ese identificador.

Ejemplo:

Cookie: session_id=3a8f92ab92

El servidor consulta la sesión para verificar al usuario.

Ventajas

- Fácil de invalidar
- Más control desde el servidor

Desventajas

- Requiere almacenar sesiones
- Menos escalable en arquitecturas distribuidas

OAuth

OAuth es un **protocolo de autorización que permite iniciar sesión usando cuentas externas**.

Ejemplos:

- Login con Google

- Login con Facebook
- Login con GitHub

Flujo básico:

1. Usuario hace clic en "Iniciar sesión con Google".
2. Se redirige al servidor de Google.
3. Google autentica al usuario.
4. Google envía un token al sistema.
5. El sistema usa ese token para identificar al usuario.

Esto evita que el sistema tenga que manejar contraseñas directamente.

3. ¿Cuándo conviene usar tokens y cuándo sesiones?

Usar sesiones cuando:

- Aplicaciones tradicionales
- Backend monolítico
- Aplicaciones pequeñas o medianas
- Seguridad más controlada

Ejemplo:

Panel administrativo.

Usar JWT cuando:

- APIs REST
- aplicaciones SPA (React / Next.js)
- microservicios
- sistemas distribuidos

Ejemplo:

Aplicación web con frontend separado del backend.

Comparación

Característica	JWT	Sesiones
Escalabilidad	Alta	Media
Control del servidor	Bajo	Alto
Invalidación inmediata	Difícil	Fácil
Uso común	APIs modernas	aplicaciones tradicionales

4. Riesgos de seguridad en un mal diseño de autenticación

Un sistema mal diseñado puede generar vulnerabilidades graves.

Robo de tokens

Si un atacante obtiene un token JWT puede usarlo hasta que expire.

Ataques de fuerza bruta

Intentar muchas contraseñas hasta acertar.

Mitigación:

- límite de intentos
- captcha
- bloqueo temporal

Secuestro de sesión

Un atacante roba el session ID del usuario.

Mitigación:

- cookies seguras
- HTTPS
- expiración de sesión

Escalamiento de privilegios

Un usuario obtiene permisos de administrador.

Ejemplo:

Modificar manualmente el rol en una petición.

Mitigación:

- validar roles en backend
- no confiar en datos del cliente

Falta de expiración

Sesiones o tokens sin expiración permiten acceso permanente.

5. ¿Cómo se definen roles y permisos en sistemas reales?

Los sistemas suelen usar **RBAC (Role Based Access Control)**.

RBAC significa **control de acceso basado en roles**.

Estructura:

Usuario -> Rol -> Permisos

Ejemplo:

Usuario	Rol
Juan	cliente
Ana	empleado
Luis	administrador

Roles

Ejemplo:

cliente
empleado
administrador

Permisos

Ejemplo:

ver_productos
crear_pedidos
editar_pedidos
eliminar_pedidos
gestionar_usuarios

Relación

Rol	Permisos
cliente	ver_productos
empleado	ver_productos, crear_pedidos
admin	todos

Diseño del sistema

Contexto del sistema

El sistema permitirá controlar el acceso a un fraccionamiento mediante **códigos QR**, donde los visitantes o repartidores podrán ingresar solo si su acceso es validado por el sistema.

El proceso general será:

1. Un **repartidor** llega al fraccionamiento.
2. Presenta su **código QR de acceso**.
3. El **guardia escanea el QR**.
4. El sistema valida la información.
5. Si el acceso es válido, el sistema permite el ingreso.

El sistema también contará con un **panel administrativo** para gestionar accesos, usuarios y registros.

Tipos de usuarios

Repartidor

Es un visitante temporal que accede al fraccionamiento para realizar entregas.

Permisos:

- Generar o recibir QR de acceso
- Mostrar QR al llegar
- Consultar estado de su acceso

Guardia

Es el personal encargado de controlar la entrada al fraccionamiento.

Permisos:

- Escanear QR
- Validar accesos
- Registrar entradas y salidas
- Ver historial de accesos recientes

Administrador

Es el usuario con control completo del sistema.

Permisos:

- Crear usuarios
- Gestionar guardias
- Configurar accesos
- Ver reportes
- Administrar el sistema

Arquitectura general del sistema

Aplicación del repartidor

Permite:

- generar o mostrar QR
- consultar acceso autorizado
- recibir confirmación de acceso

Aplicación del guardia

Permite:

- escanear QR
- validar accesos
- registrar entradas
- visualizar alertas

Panel administrativo

Permite:

- gestionar usuarios
- configurar permisos
- monitorear accesos
- generar reports

Backend API

Es el **núcleo del sistema**.

Responsabilidades:

- manejar autenticación
- validar tokens

- procesar solicitudes
- verificar permisos

Servicio de autenticación

Controla:

- login
- generación de tokens JWT
- verificación de identidad

Servicio de validación QR

Se encarga de:

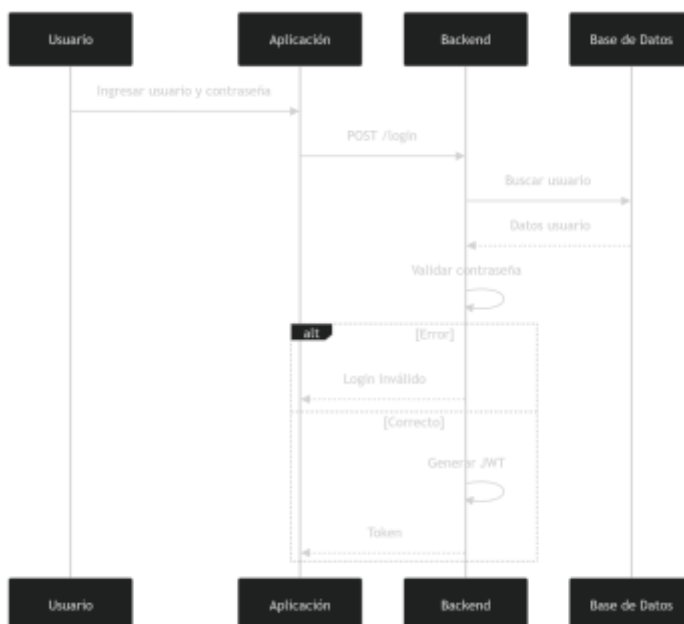
- verificar que el QR sea válido
- comprobar que no esté expirado
- confirmar que el acceso esté autorizado

Servicio de control de acceso

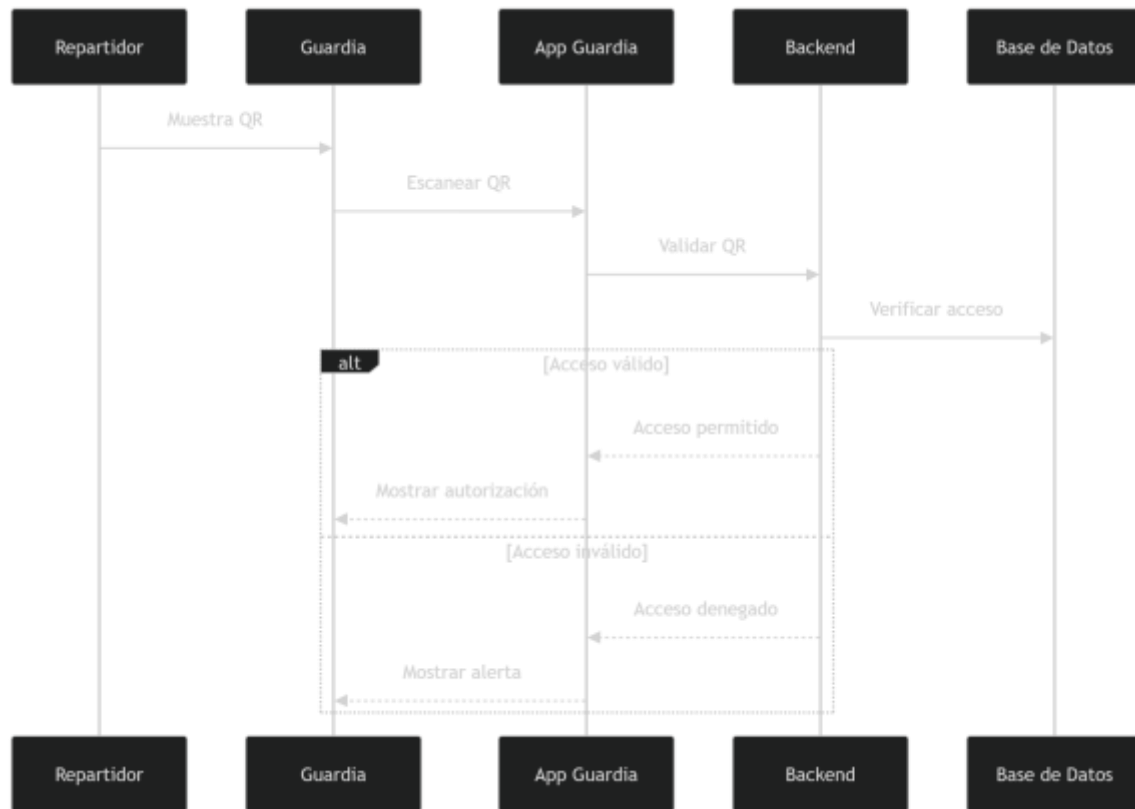
Controla:

- quién entra
- cuándo entra
- registro de accesos

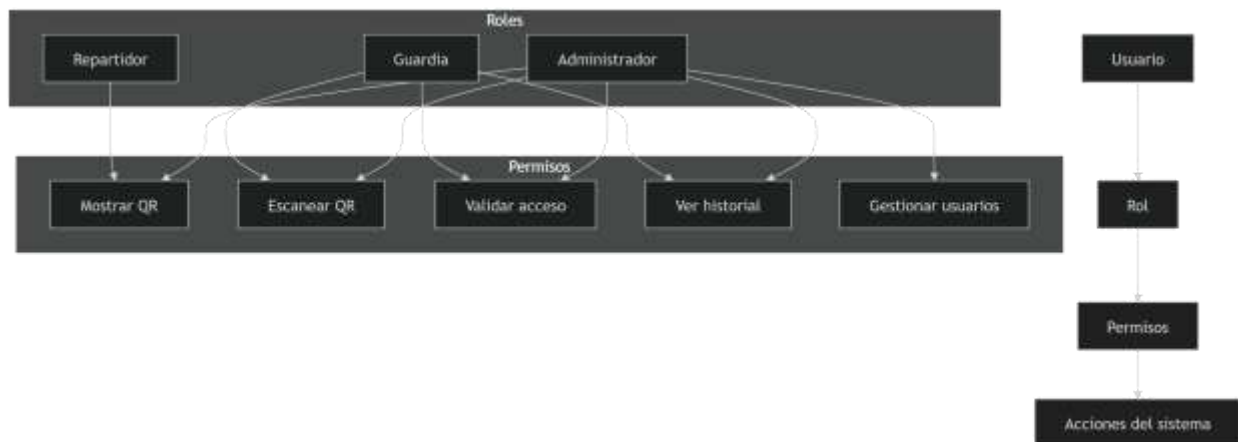
Flujo de autenticación del sistema



Flujo de acceso con QR

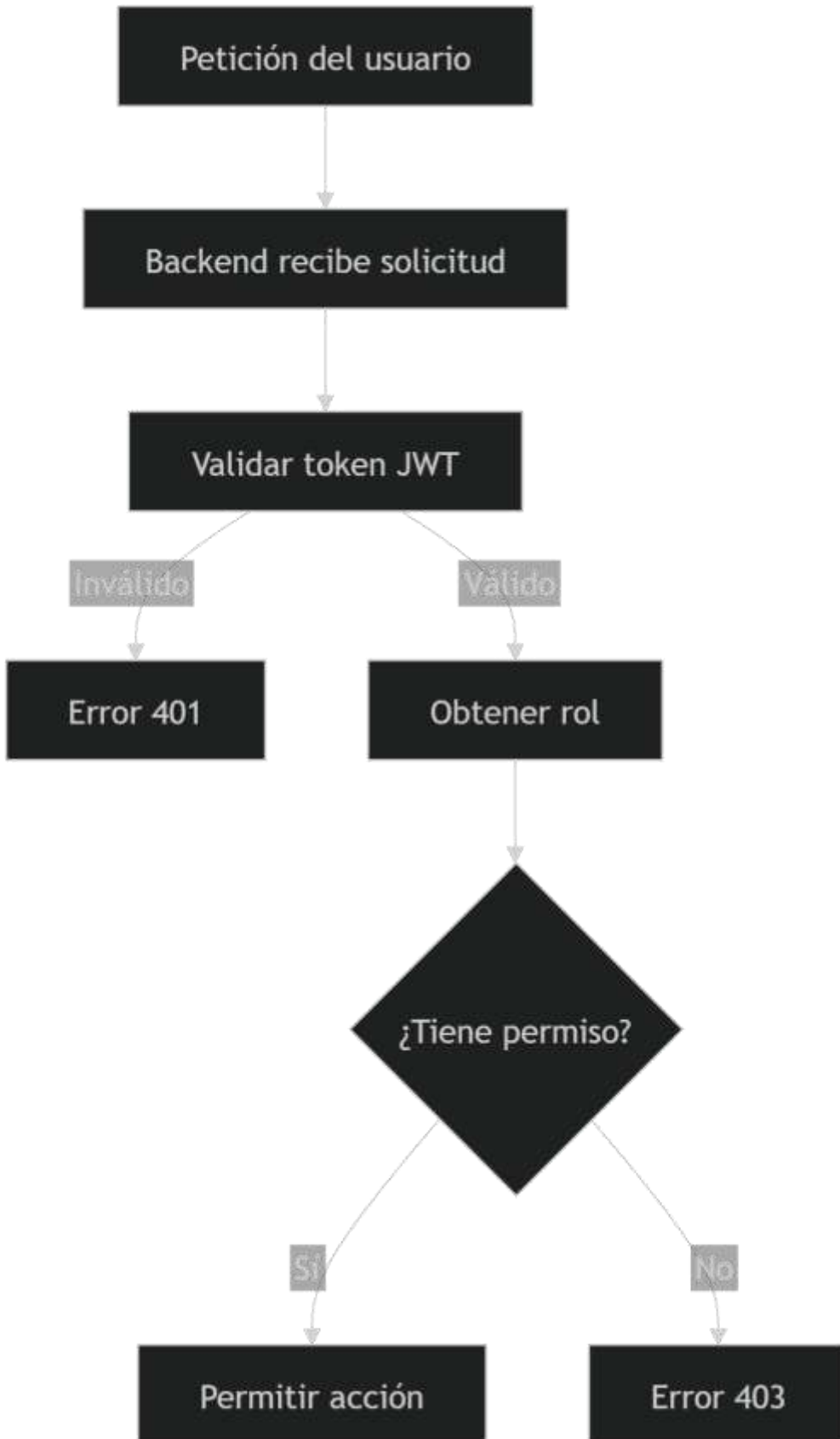


Modelo de roles y permisos (RBAC)



Flujo de autorización del sistema

Cada vez que un usuario realiza una acción:



Base de datos conceptual

Tablas principales:

Usuarios

usuarios

id

nombre

email

password

rol

Accesos QR

accesos_qr

id

codigo_qr

usuario_id

fecha_creacion

fecha_expiracion

estado

Registros de acceso

registro_accesos

id

usuario_id

guardia_id

fecha

estado

Riesgos de seguridad que se previenen

Riesgo	Mitigación
QR falsificado	validación en backend
Acceso repetido	QR con expiración
Token robado	expiración JWT
Escalamiento de privilegios	validación de roles
Ataques de fuerza bruta	límite de intentos

Decisiones técnicas del diseño

Autenticación

JWT (JSON Web Token)

Motivo:

- ideal para API
- escalable
- fácil integración con apps móviles

Control de acceso

RBAC (Role Based Access Control)

Motivo:

- control claro de permisos
- fácil mantenimiento
- escalable

Conclusión

El sistema propuesto permite controlar el acceso al fraccionamiento de manera segura mediante el uso de códigos QR y autenticación basada en tokens.

La arquitectura separa claramente:

- autenticación
- validación de accesos
- control de permisos

lo que permite construir un sistema seguro, escalable y fácil de mantener.